

Onyx — Project Overview

Onyx · multi-process OS on Raspberry Pi 4

1. In one sentence

Onyx is a **small multi-process operating system** for the **Raspberry Pi 4** (AArch64), built **on top of Circle** ([rsta2/circle](#)), which serves as its HAL and driver stack. It loads **ELF programs from the SD card** and runs them as **applications isolated from one another**, each in its own page table. The whole thing is driven by the **Onyx desktop** made up of a software compositor, a taskbar, a launcher, a terminal, a file manager, and about thirty applications.

It **runs on real Raspberry Pi 4 hardware** (not just in emulation).



The Onyx

desktop, a modernised CDE: the menu bar, the agenda on the Voronoi wallpaper, a calculator behind (grey frame), the terminal in front (its frame in the theme's colour), the dock with its Internet drawer open. (The real apps, run on a PC by the desktop simulator.)

2. Philosophy: "own the top, reuse the bottom"

Circle is a *single-process, single-address-space, cooperative* bare-metal framework ("your application **is** the system"). Onyx **keeps the bottom half** of Circle (which is excellent) and **replaces its "operating system" half**:

Layer	Decision	Detail
Reset → EL1, MMU activation, GIC, C++ constructors	Keep Circle	Our entry point is <code>main()</code> ; everything below it is already done.
64 KB page allocator (<code>palloc/pfree</code>)	Keep — it's our frame allocator	<code>CMemorySystem::Get()</code> .
GIC-400, timer, mailbox, EMMC+FatFs, USB, Ethernet, framebuffer	Reuse	This is the whole point of building on Circle.
<code>CScheduler/CTask</code> scheduler	Rewrite the implementation, keep the API	See kernel internals .
Per-process page tables, context switching, exception vectors, address spaces	Write from scratch	Doesn't exist in Circle.

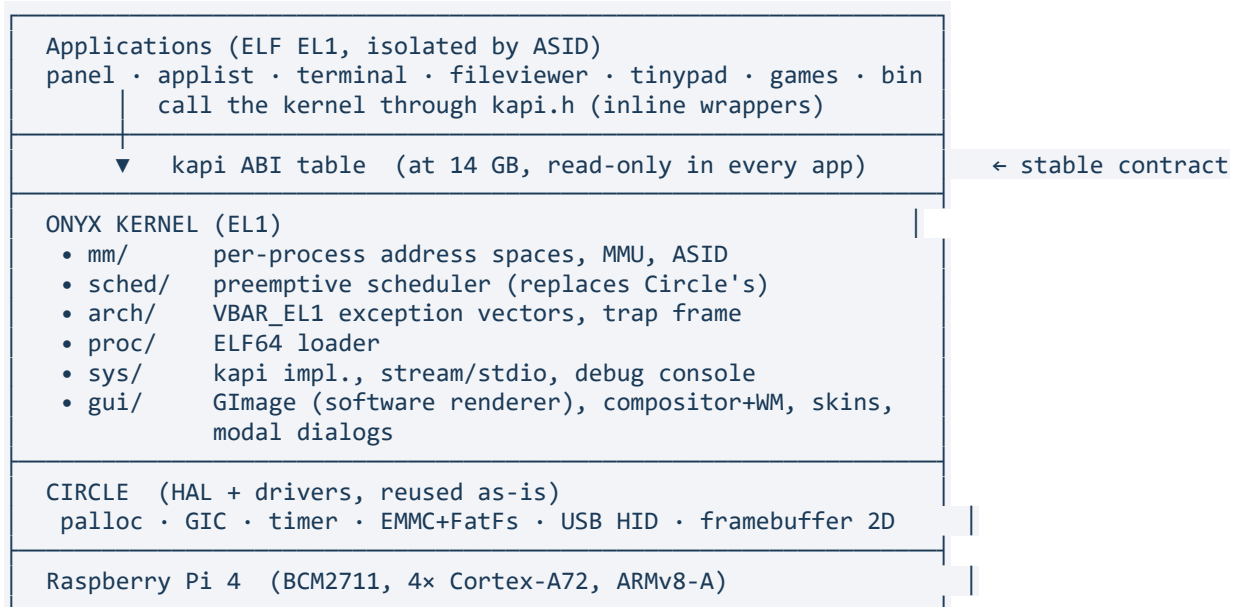
Practical consequence: we **link** the kernel against Circle's already-compiled static libraries (`libcircle.a`, `libfs`, `libusb`, ...) rather than copying its sources.

3. Main features

- **Per-process isolation via the MMU.** Each application has its own L2/L3 page table (64 KB granule), tagged by **ASID**. One application cannot see another's memory.
- **"Option C" execution model.** Applications run in **EL1** (privileged) and call the kernel's functions directly (no `SVC` trap in the common path). The trade-off: apps are isolated **from one another**, but **not from the kernel**. See §5.
- **Stable fixed-address ABI (`kapi`).** The kernel publishes a **table of function pointers** at a fixed virtual address, mapped read-only into each application. Applications call the kernel through this table → **an application binary keeps working without recompilation** when the kernel changes (*append-only* contract, current ABI version: **21**).
- **Full graphical desktop.** 32-bit software compositor, window manager, toolkit of kernel-drawn widgets (buttons, checkboxes, sliders, text fields, scroll bars, icons...), windows with themeable decoration, wallpaper, mouse cursor.
- **Onyx shell.** Panel/launcher (panel), app list, interactive terminal with **pipes and redirection** (`|`, `>`, `>>`, `<`), file manager, and a collection of command-line tools in `/bin`.
- **Application catalog.** Text editor, spreadsheet, scientific calculator, drawing program, fractal browser, calendar, task manager, theme editor, and many games (Tetris, Snake, 2048, Minesweeper, Sokoban, Pong, Game of Life, SameGame).
- **USB input devices.** Keyboard and mouse (HID), with hot-swappable keyboard layout switching (US, UK, DE, FR, ES, IT, Dvorak).
- **Networking (WLAN).** TCP/IP stack + on-board Wi-Fi (BCM4343 / `wpa_supplicant`) brought up on the primary core, TCP sockets exposed to apps through the ABI, an **IRC client**, and NTP clock synchronisation.
- **A modern web browser.** NetSurf, ported to Onyx: `http://` and `https://`, **CSS3** (`calc()`, `var()`, `flexbox`, `grid`, `gradients`, `shadows`, `rounded corners`, `gradient text`, `vendor prefixes`), web fonts

(WOFF/WOFF2, variable fonts) with Chrome's Windows fonts stood in by metric-compatible ones, and **JavaScript** on QuickJS (ES2023, the DOM, the page laid out again after a script's changes). Details in [the NetSurf changes](#). (A dedicated network core is a planned next step.)

4. The architecture at a glance



5. The "Option C" execution model

This is the most structurally defining architectural decision. After prototyping EL0 processes + system calls (SVC), the project switched to **Option C**:

- Applications run in **EL1** (the same privilege level as the kernel), **each in its own page table** tagged by ASID.
- They **call the kernel's functions directly** via the `kapi` ABI table — no system trap in the normal path.
- **Isolation**: applications are isolated **from one another** (a distinct ASID/TTBR0 per app; an app cannot address another's pages). The **kernel is not protected**: an EL1 app can technically touch the kernel's memory. This is the accepted trade-off in exchange for the ergonomics of direct calls.
- The EL0/SVC machinery (vectors, system-call dispatch) **still exists but is inert**; it could be reused if one wanted true EL0 applications isolated from the kernel.

6. Scheduling: preemptive applications, non-preemptive kernel

A **100 Hz timer tick** gives each task a time slice (20 ms); when it expires, the kernel **preempts the application** and runs the next task. An application stuck in a loop that never yields no longer freezes the system: the cursor, the desktop and the other apps stay responsive, and `taskman` can stop it (the `spin` app tests this).

Preemption from the IRQ is not straightforward in Circle's model: threads run in **EL1t** (with `SP_EL0`), while the IRQ handler runs in **EL1h** (with `SP_EL1`), so a context switch inside the IRQ would swap the wrong stack. The kernel therefore **redirects the IRQ's return** into a small trampoline that runs on the app's own stack, saves its full register and FP state, and yields like a voluntary switch.

Only **application code** is preempted: an app inside a `kapi_*` call, and the kernel's own threads, are not (the **kernel is non-preemptive**, the classic Unix model); its long operations (file reads and writes, SD-card waits) yield between pieces. Tasks still also switch **voluntarily** (`yield`, `msleep`, `present`, `wait`, ...), and a dynamic priority keeps CPU hogs from starving the tasks that yield (the network, the sound). Details in [Kernel internals](#) (§ "Preemptive scheduling").

An application may run **threads** (kapi v67): more tasks in its own address space, with mutexes, events and barriers, and calls posted to its event pump — a thread waits on the network while the window stays responsive. The process ends with its main thread; killing it kills them all. The scheduler's task list has no fixed limit.

7. Target hardware

- **Raspberry Pi 4** (BCM2711, 4× Cortex-A72, ARMv8-A) — primary target.
- **HDMI** output (32 bpp framebuffer, 1024×768 by default, configurable).
- **USB keyboard + mouse**.
- Optional **serial console** (GPIO14/15, 115200 8N1) for the boot log and exception dumps.
- The RPi 5 is deferred (I/O behind the proprietary RP1 chip via PCIe — heavier bring-up). Multi-core (SMP) is also deferred.

8. Repository structure

ARCHITECTURE.md	original design + build manifest (historical, partly dated)
README.md	summary (partly dated: describes the "2 demos" state)
docs/	THIS documentation (overview, internals, dev/user guides)
kernel/	the Onyx kernel (see docs/02-KERNEL-INTERNALS.md)
user/	the userland: apps (*.c), runtime (crt0.S, user.ld), /bin tools
sdcard/	ready-to-flash files for an RPi 4 (firmware, config, apps, /bin)
tools/	host scripts (BMP icon generation)
circle/	upstream Circle clone (not committed; cloned separately)

Beware of legacy docs. ARCHITECTURE.md, README.md, kernel/README.md, and sdcard/README.md describe **older states** of the project (ELO processes, preemptive scheduling, 640×480, "two demos"). Where they contradict this documentation, **these documents here (docs/) are authoritative** for the current state. ARCHITECTURE.md §11–§12 nevertheless remains the best reference for *why* Option C was chosen (its "cooperative scheduling" has since been replaced by the preemption of §6).

9. Further reading

Document	For whom	Contents
02 — Kernel Internals	anyone who wants to understand the kernel	boot, memory/MMU, scheduling, exceptions, ABI, GUI, streams
03 — Developer Guide	anyone who wants to build/compile/extend	toolchain, build, app model, extending the ABI, conventions, debugging
04 — User Guide	anyone who wants to use it	SD card, desktop, terminal, files, applications, customization

10. Quick genesis (milestones)

1. Taking control of `main()` on top of Circle's `sysinit`; serial console.
2. Replacement scheduler ("shadow" header for `CScheduler`).
3. `VBAR_EL1` vectors, trap frame, system-call path.
4. Per-process page tables (`CAddressSpace`), TTBR0/ASID switching.
5. ELF64 loader → process.
6. Framebuffer (`C2DGraphics`) + `GImage` rendering core (ported from the author's FreeBASIC `SimpleOS`).
7. Compositor + window manager; two animated demos running simultaneously.
8. **Switch to Option C** (EL1 apps + direct call) then **fixed-table ABI**.
9. Onyx desktop (panel + applist), stream/stdio subsystem, terminal + `/bin`, file manager.
10. Modal dialogs, themes, app-drawn wallpaper, PID management, keyboard layouts, theme editor (ABI v16).
11. Networking: WLAN bring-up + TCP/IP on the primary core, TCP socket calls (ABI v21), an IRC client, a `/bin/net` tool, and NTP clock sync.